off to fi forces a boxing off since fi has type IFoo.

Thus method outputs:

```
00:00:00
    00:00:00.0468915
```

which shows that boxing and calling the method takes significantly more time than just calling the method. In general, we want to avoid boxing when (reasonably) possible to avoid these costs.

The opposite conversion, unboxing is also possible. Although, unboxing only occurs explicitly through a cast. This is because in an unboxing operation, we are casting from a more general type, e.g., object or an interface type, to the more specific struct type. This cast may fail so the programmer must state their intentions appropriately.

```
public class UnboxTest
{
    public static void Test()
    {
        int x = 42;
        object o = x;
        x = ((int)o) + ((int)o);
    }
}
```

3.3 Operator overloading, indexers, and conversions

As you've seen, C# allows operator overloading for user-defined types. This has the benefit of increasing readability as long as the operators are overloaded in a reasonable manner.

```
struct Vec
{
    public int X { get; private set; }
    public int Y { get; private set; }
    public Vec(int x, int y) : this()
    {
        X = x;
        Y = y;
    }

    /// <summary>
    /// Standard vector addition.
```